

# Data Analysis with Python 2024–2025

## Parte 5: Pandas (DataFrames na Prática)

Caderno de Exercícios

Tradução e Adaptação: The University of Helsinki MOOC Center

### Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**.

### Conteúdo

# 1 Exercícios - Pandas na Prática

## Exercício 1 – Capitais

### Sumário

- **Objetivo:** Escreva uma função `capitais()` que devolva o seguinte DataFrame, com o nome do estado como índice das linhas:

|            | Capital   | Populacao |
|------------|-----------|-----------|
| Pernambuco | Recife    | 1653461   |
| Bahia      | Salvador  | 2418616   |
| Ceara      | Fortaleza | 2428708   |
| Parana     | Curitiba  | 1963726   |
| Amazonas   | Manaus    | 2219580   |

## Exercício 2 – Tabela de potências

### Sumário

- **Objetivo:** Implemente a função `tabela_de_potencias(serie, k)`, que recebe uma Series e um inteiro positivo `k`, e devolve um DataFrame com o mesmo índice da série de entrada. A primeira coluna deve conter a própria série; a segunda, a série elevada ao quadrado; a terceira, ao cubo; e assim por diante até a potência `k`. As colunas devem ser nomeadas com os números de 1 a `k`.

- **Exemplo de uso:**

```
s = pd.Series([2, 3, 5], index=list("xyz"))
print(tabela_de_potencias(s, 3))
```

- **Saída esperada:**

|   | 1 | 2  | 3   |
|---|---|----|-----|
| x | 2 | 4  | 8   |
| y | 3 | 9  | 27  |
| z | 5 | 25 | 125 |

### Exercício 3 – Carregando um arquivo tabulado

#### Sumário

- **Objetivo:** Suponha que exista um arquivo `vendas.tsv`, separado por tabulações, com colunas de vendas mensais de uma loja. Escreva uma função `carregar_vendas(caminho)` que leia o arquivo com `pd.read_csv` (lembrando de indicar o separador correto) e imprima, no seguinte formato, o número de linhas/colunas e os nomes das colunas:

```
Formato: linhas, colunas
Colunas:
col1
col2
...
```

### Exercício 4 – Filtro de funcionários

#### Sumário

- **Objetivo:** Dado um DataFrame `funcionarios` com colunas `nome`, `cargo`, `salario` e `setor`, escreva a função `salarios_altos(df, limite)` que devolva apenas as linhas cujo salário é maior que `limite`, contendo somente as colunas `nome` e `salario`.

### Exercício 5 – Seleção com loc

#### Sumário

- **Objetivo:** Escreva a função `selecionar_com_loc(df)` que, usando obrigatoriamente o atributo `loc`, devolva as linhas de índice "Ana" até "Diego" (incluindo ambos), restritas às colunas "salario" e "setor".

### Exercício 6 – Seleção com iloc

#### Sumário

- **Objetivo:** Escreva a função `top5_por_posicao(df)` que devolva as 5 primeiras linhas e as duas primeiras colunas de um DataFrame, usando uma única chamada ao atributo `iloc`.

### Exercício 7 – Temperatura máxima

#### Sumário

- **Objetivo:** Escreva a função `temperatura_maxima(df)` que recebe um DataFrame de medições climáticas diárias (com uma coluna "Temperatura (C)") e devolve o valor máximo registrado. No programa principal, imprima o resultado no formato: Temperatura maxima: xx.x.

## Exercício 8 – Média de um mês específico

### Sumário

- **Objetivo:** Escreva a função `media_do_mes(df, mes)` que devolva a temperatura média do mês informado (um número de 1 a 12), assumindo que o DataFrame tenha uma coluna "mes".

## Exercício 9 – Dias de chuva

### Sumário

- **Objetivo:** Escreva a função `dias_de_chuva(df)` que devolva o número de dias em que a coluna "Precipitacao (mm)" foi maior que zero.

## Exercício 10 – Limpeza de um sensor com falhas

### Sumário

- **Objetivo:** Um arquivo `sensor.csv` contém leituras horárias de um sensor de umidade, mas as últimas linhas do arquivo estão totalmente vazias (erro de exportação), e existe uma coluna extra que ficou inteiramente sem valores. Escreva a função `limpar_sensor(caminho)` que carregue o arquivo, remova as linhas totalmente vazias e remova também as colunas totalmente vazias, devolvendo o DataFrame limpo.

## Exercício 11 – Tipos de valores ausentes

### Sumário

- **Objetivo:** Escreva a função `tabela_com_lacunas()` que reconstrua o seguinte DataFrame, usando a coluna `Pais` como índice. A coluna `Fundacao` deve ter tipo `float` (para admitir valores ausentes) e a coluna `Capital` deve ter tipo `object`. Onde houver um travessão, use o valor de ausência apropriado.

| Pais      | Fundacao | Capital      |
|-----------|----------|--------------|
| Brasil    | 1822     | Brasilia     |
| Argentina | 1816     | Buenos Aires |
| Chile     | -        | Santiago     |
| Uruguai   | 1825     | -            |
| Paraguai  | 1811     | Assuncao     |

## Exercício 12 – Separando nome completo e normalizando texto

### Sumário

- **Objetivo:** Escreva a função `separar_nomes(serie)` que recebe uma Series de nomes completos escritos em letras minúsculas (por exemplo, "joana pereira") e devolve um DataFrame com duas colunas, "Primeiro\_nome" e "Sobrenome", ambas com a primeira letra maiúscula. Assuma que cada nome tem exatamente duas palavras.

**Créditos:** Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.